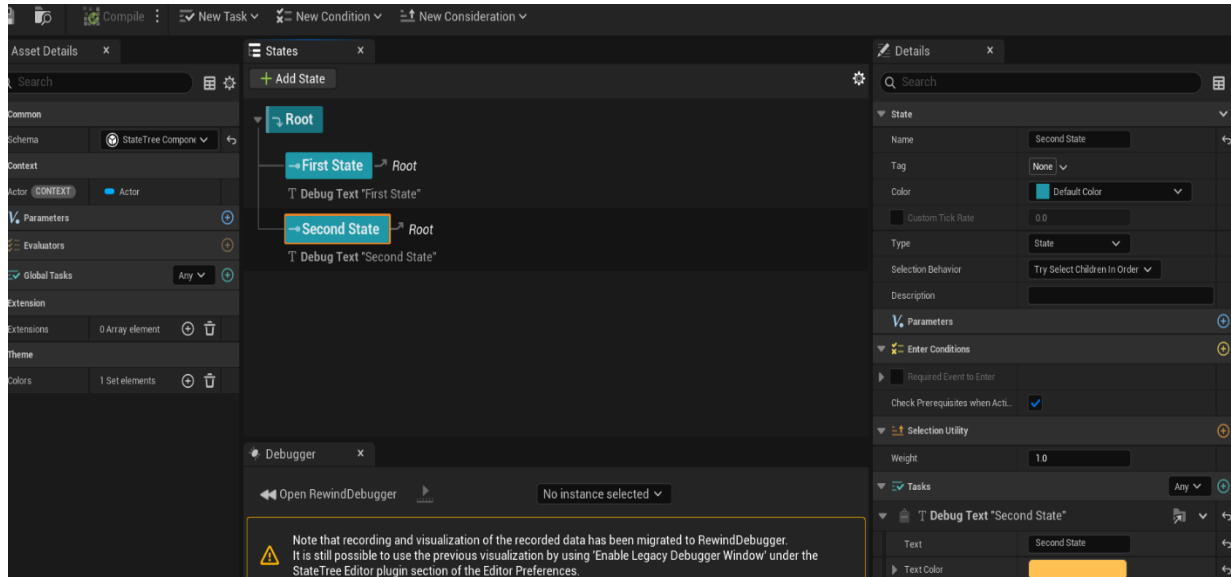


Task 1: Basic State Tree Setup

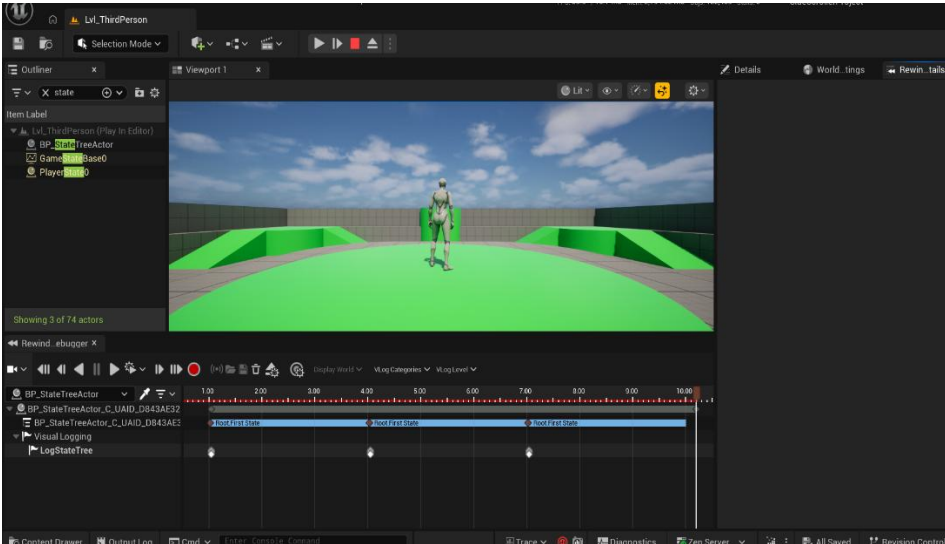
The first step was creating a state tree asset and attaching it to an actor via a StateTreeComponent. I created a new BP_StateTreeActor, added the state tree Component, and pointed it at the state tree asset. Inside the state tree, I added two child states under root: First State and Second State. Each state got a Debug Text task and a Delay task set to 3 seconds.

At this point, the state tree just loops on the first state since there are no transitions telling it to move to Second State.



Task 2: Rewind Debugger

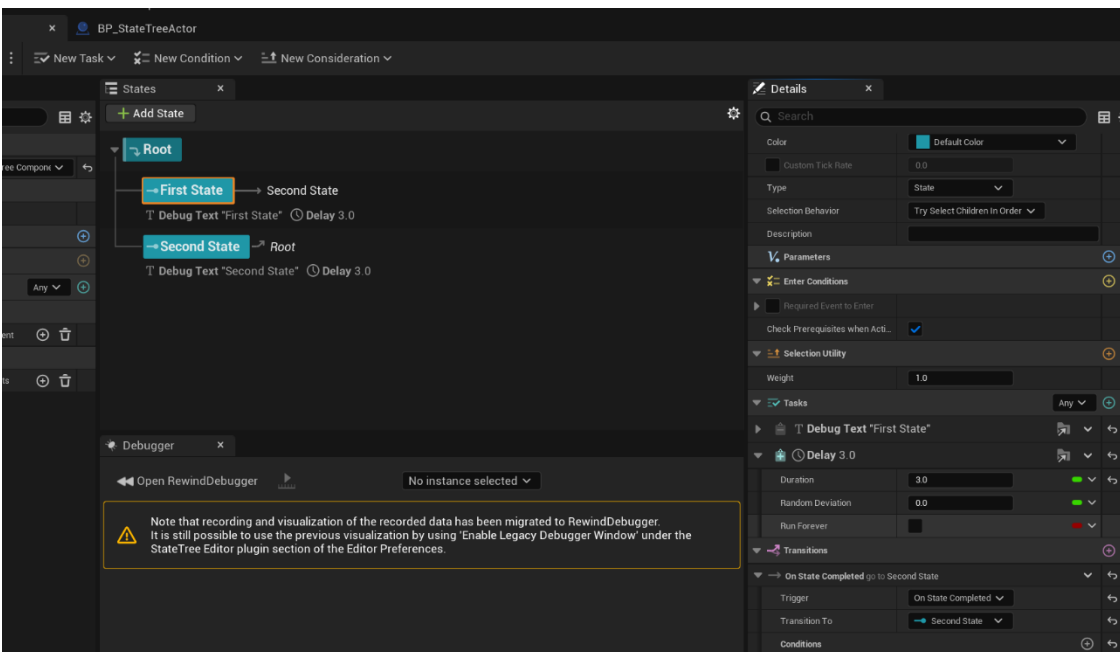
With the Actor placed in the level, I ran the editor in PIE mode and opened the Rewind Debugger from within the state tree Editor. After selecting the BP_StateTreeActor instance and hitting record, the debugger timeline showed the state tree execution in real time. As expected, it showed First State running on repeat since no transitions had been set up yet.



Task 3: Transitions and Conditions

Part A: Transitions

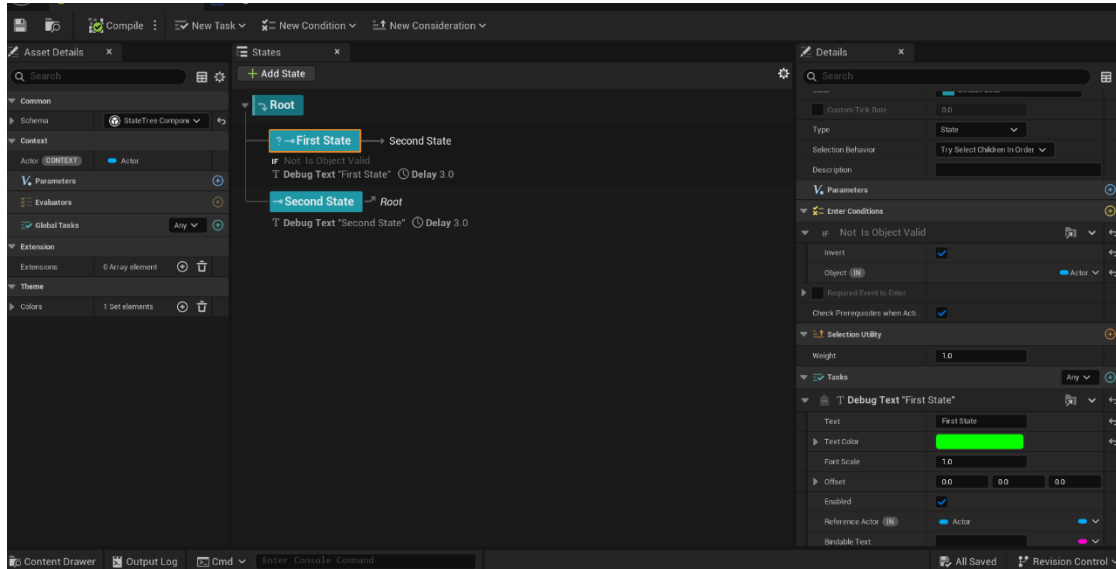
I added a Transition on First State set to trigger "On State Completed" and point to Second State. After running this in PIE, the debugger timeline showed the state tree executing First State followed by Second State in sequence, confirming the transition was working as intended.



Part B: Forced Condition Failure

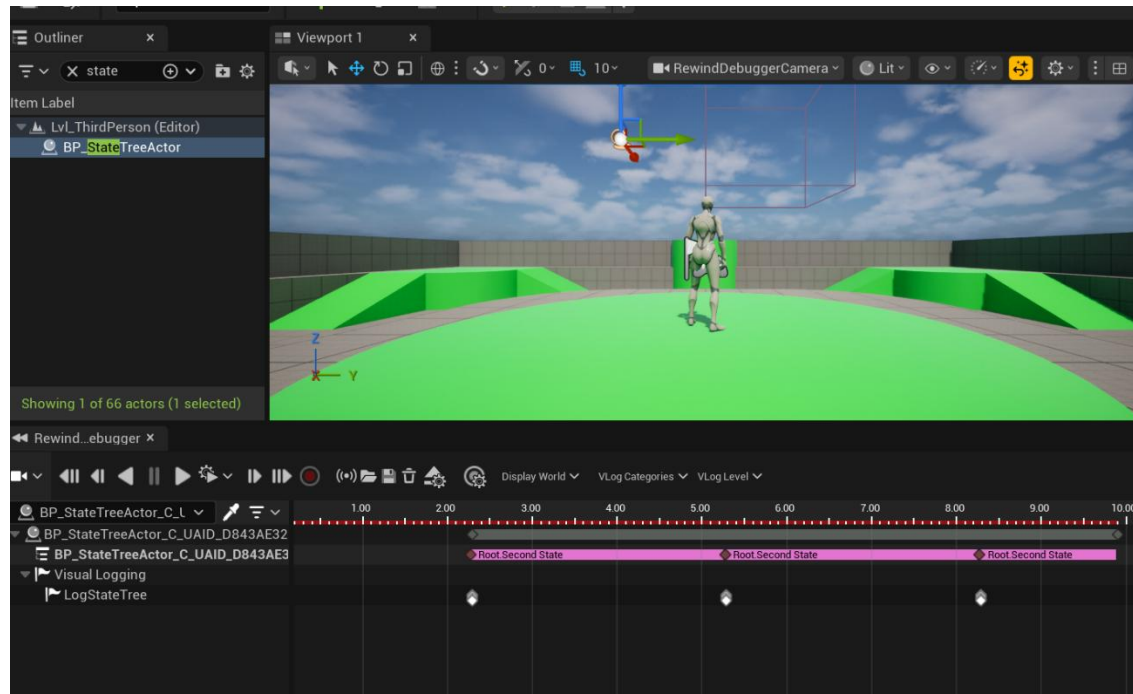
For this part, I added an Enter Condition on first state using an inverted "Is Object Valid". This forces the condition to always fail, meaning the first state never gets entered. The state tree skips it entirely and goes straight to the second state.

Setup:



Running this in the debugger confirmed it, the timeline only showed Second State executing on each cycle, with no trace of First State being entered at any point.

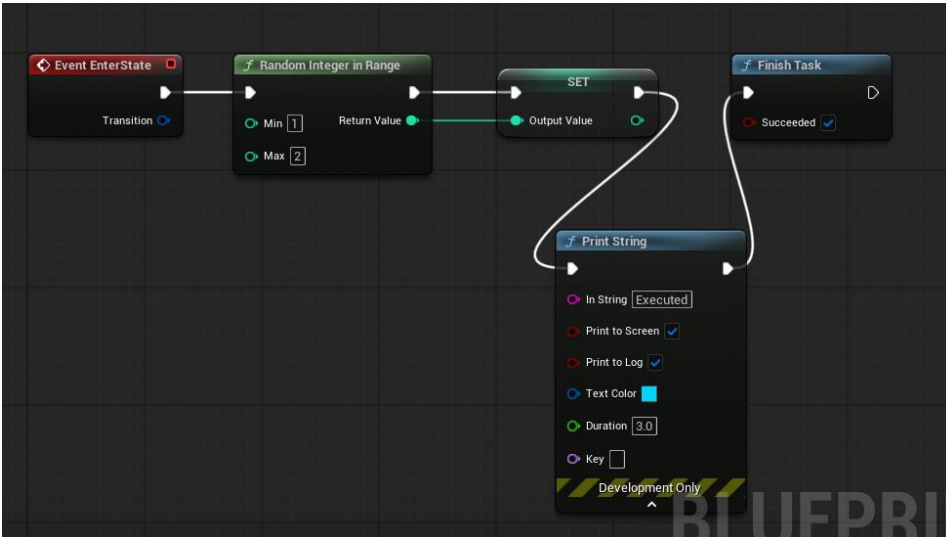
Debugger Result:



Task 4: Custom Task with Random State Selection

This task required creating a custom state tree Task that generates a random integer and uses it to determine which state gets entered. I started by adding an Int parameter called RandomValue at the state tree level.

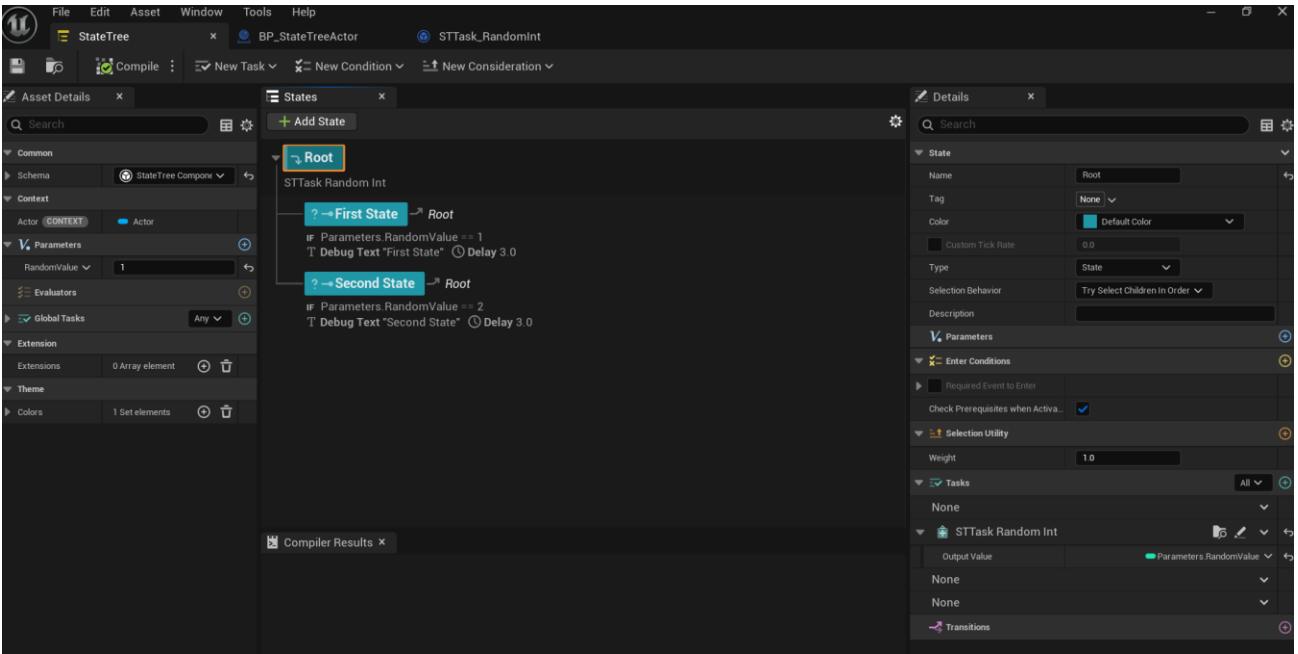
I then created a new STTask_RandomInt. In its EnterState event, I used a Random Integer in Range node (Min 1, Max 2) to generate a value, stored it in an exposed output variable, and finished the task.



Back in the state tree, I initially placed the task on the root, but ran into an issue where the RandomValue wasn't being set before the child states evaluated their Enter Conditions. Moving it to global resolved this, but global only ran once at startup so the value never re-rolled. I moved it back to root and the conditions evaluated correctly from there. The states completed instantly rather than waiting for the delay, which I could not find the reason as to why they would no longer run their delays, as I changed nothing related to that.

First State has an Enter Condition checking `Parameters.RandomValue == 1`, and the Second State checks `Parameters.RandomValue == 2`. The task's Output Value is bound to `Parameters.RandomValue`. Running this in PIE and recording with the Rewind Debugger showed the state tree randomly selecting between the two states on each cycle.

Final setup:



As seen in the final result we are very rapidly changing between states:

